Project Directory and Flowchart for Data-Driven Media Optimization

1. Project Directory Setup for GitHub

The GitHub repository organizes the project's assets logically, ensuring learners can manage code, data, models, visualizations, and documentation efficiently. The structure is designed for collaboration, reproducibility, and alignment with the project's deliverables (e.g., models, Power BI dashboards, Streamlit app). Below is the directory layout with descriptions.

```
Data-Driven-Media-Optimization/
├── data/
│   ├── raw/
│   │   ├── reviews.json              # Raw Yelp reviews data
│   │   ├── business.json             # Raw Yelp business data
│   │   ├── photos.json               # Raw Yelp photos metadata
│   │   └── photos/                   # Folder for raw image files
│   ├── processed/
│   │   ├── sentiment_data.csv        # Cleaned reviews for NLP
│   │   ├── timeseries_data.csv       # Aggregated data for TSA
│   │   └── image_data.csv            # Image paths and labels for CV
│   └── external/
│       └── README.md                 # Notes on external data sources
├── src/
│   ├── data_preparation/
│   │   ├── preprocess_reviews.py     # Script for NLP data cleaning
│   │   ├── preprocess_timeseries.py  # Script for TSA data aggregation
│   │   └── preprocess_images.py      # Script for CV image processing
│   ├── modeling/
│   │   ├── nlp_model.py              # NLP sentiment analysis model
│   │   ├── tsa_model.py              # TSA forecasting model
│   │   └── cv_model.py               # CV image classification model
│   ├── visualization/
│   │   ├── eda_notebook.ipynb        # EDA visualizations
│   │   └── powerbi_dashboard.pbix    # Power BI dashboard file
│   └── app/
│       ├── app.py                    # Streamlit app script
│       └── requirements.txt          # Streamlit dependencies
├── models/
│   ├── nlp_model.pkl                 # Trained NLP model
│   ├── tsa_model.pkl                 # Trained TSA model
│   └── cv_model.h5                   # Trained CV model
```

```
├── docs/
│   ├── project_charter.md           # Project charter
│   ├── data_dictionary.md           # Data dictionary
│   ├── eda_report.md                # EDA report
│   ├── model_evaluation/
│   │   ├── nlp_evaluation.md        # NLP model performance
│   │   ├── tsa_evaluation.md        # TSA model performance
│   │   └── cv_evaluation.md         # CV model performance
│   ├── final_report.md              # Final project report
│   └── presentation/
│       └── final_presentation.pptx  # Final presentation slides
├── notebooks/
│   ├── exploratory_analysis.ipynb   # Jupyter notebook for initial EDA
│   ├── nlp_model_training.ipynb     # Notebook for NLP modeling
│   ├── tsa_model_training.ipynb     # Notebook for TSA modeling
│   └── cv_model_training.ipynb      # Notebook for CV modeling
├── .gitignore                       # Ignores large data files, models
├── README.md                        # Project overview, setup instructions
├── requirements.txt                 # Project dependencies
└── LICENSE                          # MIT license for open-source
```

Directory Descriptions

- **data/**: Stores raw and processed data.
  - **raw/**: Original Yelp dataset files (JSON) and images, not versioned in GitHub due to size (listed in `.gitignore`).
  - **processed/**: Cleaned datasets (CSV) for modeling, versioned for reproducibility.
  - **external/**: Placeholder for additional data if needed (e.g., FER2013), with documentation.
- **src/**: Contains Python scripts for data preparation, modeling, visualization, and the Streamlit app.
  - **data_preparation/**: Scripts to clean and preprocess data for NLP, TSA, and CV.
  - **modeling/**: Scripts to train and save deep learning models.
  - **visualization/**: Notebooks and Power BI files for EDA and dashboards.
  - **app/**: Streamlit app code and dependencies.
- **models/**: Stores trained model files (e.g., `.pkl`, `.h5`) for reuse in the Streamlit app.
- **docs/**: Houses documentation, including the charter, data dictionary, reports, and presentation.
  - **model_evaluation/**: Individual reports for model performance metrics.
- **notebooks/**: Jupyter notebooks for exploratory and modeling tasks, encouraging experimentation.

- **.gitignore**: Excludes large files (e.g., `data/raw/`, `*.h5`) to keep the repository lightweight.
- **README.md**: Guides users on project setup, dependencies, and execution.
- **requirements.txt**: Lists Python libraries (e.g., pandas, tensorflow, streamlit).
- **LICENSE**: Uses MIT license for clarity on usage rights.

GitHub Setup Instructions

1. **Initialize Repository**:
    - Create a repository named `Data-Driven-Media-Optimization` on GitHub.
    - Clone locally: `git clone <repo-url>`.
2. **Create Directory Structure**:
    - Use the above layout to create folders and placeholder files.
    - Example: `mkdir -p data/raw data/processed src/data_preparation`.
3. **Add Initial Files**:
    - Create `README.md` with project overview.
    - Add `.gitignore` with entries like `data/raw/*`, `*.h5`, `*.pbix`.
    - Include `requirements.txt` with:

```
pandas
numpy
tensorflow
nltk
opencv-python
streamlit
powerbiclient
```

4. **Commit and Push**:
    - `git add .`
    - `git commit -m "Initialize project directory structure"`
    - `git push origin main`
5. **Collaboration**:
    - Use branches (e.g., `feature/nlp-model`) for tasks.
    - Assign issues for tasks (e.g., “Preprocess reviews data”).
    - Review pull requests weekly to merge deliverables.

Notes for Learners

- **Data Management**: Store raw Yelp data locally or on cloud storage (e.g., Google Drive), referencing paths in scripts.

- **Version Control**: Commit processed data (`data/processed/`) and scripts, but not raw data or large models.
- **Reproducibility**: Document environment setup in `README.md` (e.g., Python version, library installation).
- **Collaboration**: Use GitHub issues to track tasks (e.g., Task 1: Project charter) and milestones for weekly deadlines.

2. Project Flowchart

The flowchart visualizes the project's workflow, mapping data flow, tasks, and deliverables across the CRISP-DM phases over 8 weeks. It integrates inputs (Yelp dataset), processes (data preparation, modeling), and outputs (dashboards, app), clarifying how components connect.

Flowchart Description

```
[Start: Project Initiation]
  |
  v
[Week 1: Business Understanding]
  - Input: E3 Media Crafters' needs
  - Process: Define objectives (sentiment analysis, forecasting, image classification)
  - Output: Project charter (docs/project_charter.md)
  |
  v
[Week 2: Data Understanding]
  - Input: Yelp dataset (data/raw/reviews.json, business.json, photos.json)
  - Process: EDA (src/visualization/eda_notebook.ipynb)
  - Output: Data dictionary (docs/data_dictionary.md), EDA report (docs/eda_report.md)
  |
  v
[Week 3-4: Data Preparation]
  - Input: Raw Yelp data
  - Processes:
    - NLP: Clean reviews (src/data_preparation/preprocess_reviews.py)
    - TSA: Aggregate sentiment (src/data_preparation/preprocess_timeseries.py)
    - CV: Process images (src/data_preparation/preprocess_images.py)
  - Outputs:
    - sentiment_data.csv (data/processed/)
    - timeseries_data.csv (data/processed/)
    - image_data.csv (data/processed/)
  |
  v
[Week 5: Modeling - NLP]
  - Input: sentiment_data.csv
  - Process: Train BERT/LSTM (src/modeling/nlp_model.py)
```

```
  - Output: nlp_model.pkl (models/), evaluation report (docs/model_evaluation/nlp_evalua
  |
  v
[Week 6: Modeling - TSA]
  - Input: timeseries_data.csv
  - Process: Train LSTM (src/modeling/tsa_model.py)
  - Output: tsa_model.pkl (models/), evaluation report (docs/model_evaluation/tsa_evalua
  |
  v
[Week 7: Modeling - CV]
  - Input: image_data.csv, photos
  - Process: Train CNN (src/modeling/cv_model.py)
  - Output: cv_model.h5 (models/), evaluation report (docs/model_evaluation/cv_evaluatio
  |
  v
[Week 8: Evaluation & Deployment]
  - Inputs: Model outputs, processed data
  - Processes:
    - Visualize: Create Power BI dashboard (src/visualization/powerbi_dashboard.pbix)
    - Deploy: Build Streamlit app (src/app/app.py)
    - Document: Compile reports, presentation
  - Outputs:
    - Power BI dashboard
    - Streamlit app
    - Final report (docs/final_report.md)
    - Presentation (docs/presentation/final_presentation.pptx)
  |
  v
[End: Project Completion]
```

Flowchart Explanation

- **Nodes**:
  - **Rectangles**: Represent CRISP-DM phases (e.g., Business Understanding).
  - **Ovals**: Inputs (e.g., Yelp dataset) and outputs (e.g., project charter).
  - **Diamonds**: Processes (e.g., Train BERT/LSTM).
- **Edges**:
  - Arrows show data flow (e.g., raw data → processed data → models).
  - Connect tasks across weeks (e.g., Week 2 EDA informs Week 3 preprocessing).
- **Components**:
  - **Inputs**: Yelp dataset files drive all tasks.

    - **Processes**: Data preparation splits into NLP, TSA, CV pipelines; modeling builds independent models.
    - **Outputs**: Deliverables (e.g., models, dashboards) align with directory structure.
- **Integration**:
    - NLP, TSA, and CV outputs converge in Week 8 for visualization (Power BI) and interactivity (Streamlit).
    - Final report and presentation synthesize all findings.

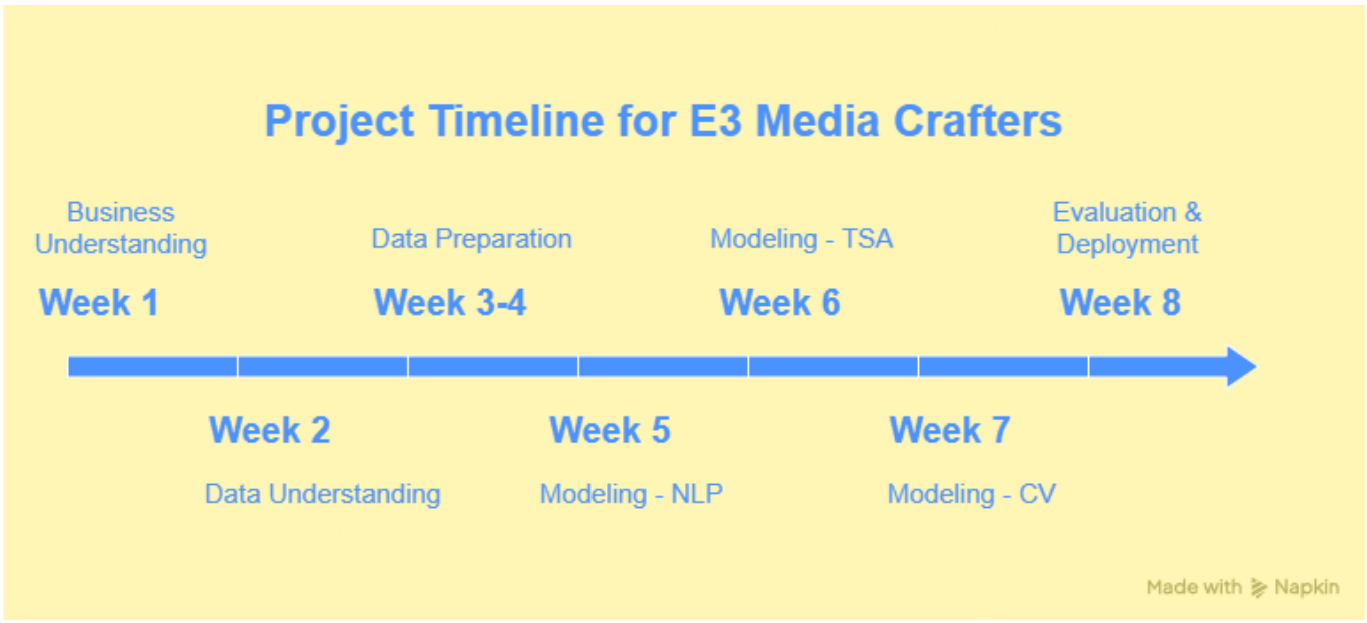
Integration with Project

- **Directory**: The structure maps to tasks (e.g., `src/data_preparation/` for Week 3-4, `models/` for Week 5-7), ensuring learners can locate files easily.
- **Flowchart**: Reflects the project charter's timeline and data dictionary's fields, guiding learners through data flow (e.g., `reviews.json` → `sentiment_data.csv` → `nlp_model.pkl`).
- **GitHub**: Encourages version control for collaboration, with issues tied to tasks (e.g., "Create nlp_model.py") and milestones for weekly deliverables.

Notes for Learners

- **Directory Use**:
    - Store raw data locally, commit only processed CSVs.
    - Run notebooks in `notebooks/` for experimentation, scripts in `src/` for production.
    - Update `docs/` weekly with deliverables.
- **Flowchart Reference**:
    - Use as a roadmap to track progress.
    - Ensure outputs (e.g., `powerbi_dashboard.pbix`) match flowchart endpoints.
- **GitHub Best Practices**:
    - Commit small changes frequently (e.g., `git commit -m "Add preprocess_reviews.py"`).
    - Use pull requests for peer reviews before merging to `main`.
- **Support**: Consult the PM during check-ins for directory or workflow issues.

This setup and flowchart provide a clear, organized framework for the project, aligning with E3 Media Crafters' goals and the internship's requirements.


Project Timeline for E3 Media Crafters
Business Understanding
Week 1
Week 2
Data Understanding
Data Preparation
Week 3-4
Week 5
Modeling - NLP
Modeling - TSA
Week 6
Week 7
Modeling - CV
Evaluation & Deployment
Week 8
Made with Napkin